

MonthlyMuse

Project Progress Report — full MVP delivered

AI-Powered Automated Message Generator for Monthly Posting · status as of October 2026 · companion to the Master Blueprint

- MVP complete
- 62 tests green
- live & demo-able
- pushed to GitHub
- deploy-ready

1. Executive summary

The project is **feature-complete against the blueprint's MVP definition (Section 15.1)** and is running live on the development machine. A user can register, define a Monthly Plan, receive three ranked message candidates with transparent score breakdowns, select/edit/reject one, watch the personalization engine update, and see the effect proven in analytics. The scheduler prepares each month's cycle automatically and sends T-3/T-1 reminders.

Verification status: 62/62 automated tests pass (including every golden value from Section 17), TypeScript typecheck and production build are clean, and the complete login → dashboard → review → feedback loop was exercised in a real browser and confirmed working end to end.

62 automated tests passing	58 API endpoints	16 database tables	12 frontend screens
14 AI pipeline stages	119 files in repository		
7.5k lines of Python	3.8k lines of TypeScript/TSX	4 runnable notebooks	40 message templates
800 tone training rows	100% MVP scope delivered		

2. Blueprint coverage

Status of every major section of the Master Blueprint.

Blueprint section	Status	What exists
§3 Product & UX (screens)	Complete	12 screens: login, register, dashboard, create, review/compare, calendar, plans, history, analytics, settings
§4 AI/NLP pipeline (14 stages)	Complete	All stages implemented in <code>app/ai/pipeline.py</code> : preprocess → context → arms → generate → hard constraints → dedup → embed → features → score → MMR → confidence → explain
§5 Mathematical core	Complete	$s = F \cdot w$, constrained gradient step, MMR, Dirichlet Monte-Carlo, Wilson interval, Thompson sampling, softmax tone classifier, chi-square, entropy
§8 Scheduler worker	Complete	APScheduler: daily cycle preparation (idempotent), T-3/T-1 reminders, catch-up, plan extension, <code>job_runs</code> audit
§9 Database schema	Complete	All 16 tables, portable string enums, UTC-normalised timestamps, float32 vector BLOBs
§10 API design	Complete	58 endpoints, uniform error envelope, JWT access + rotating refresh cookie, rate limits, request-id logging
§11 Frontend	Complete	Next.js 14 App Router, TanStack Query polling, RHF+Zod forms, Recharts visualisations
§12 Design system	Complete	Indigo token scale, component classes, reduced-motion support, responsive + mobile tab bar
§13 Architecture	Complete	Stateless API, separate worker, one database — matches the specified topology
§14 Technology stack	Complete	As specified; four documented deviations (see §6)
§15.1 MVP	Complete	Every row of the MVP table implemented and covered by tests
§16 Evaluation & analytics	Complete	Usage / Content / Quality dashboards: Wilson CIs, chi-square, novelty, entropy, MRR, top-1 agreement
§17 Golden tests	Complete	35 golden-value assertions, all passing
§19 Roadmap	Complete	All planned build stages delivered in one pass
Appendix B (repo, CI, Docker)	Complete	README, docker-compose, Dockerfiles, GitHub Actions CI, 4 notebooks
§15.2 Version 2	Not started	Platform publisher, engagement import, teams, dark mode, autopilot — intentionally out of MVP scope

3. Feature-by-feature completion

Feature	Done	Tests	Detail
Accounts & auth	100%	API journey	Register, login, logout, refresh rotation, Argon2id hashing, profile, timezone, data export, account delete
Monthly Plans	100%	API + scheduler	12-cycle creation, pause/resume/delete, auto-extension, next-generation computation
Create Post form	100%	UI verified	3-step form, draft autosave, tone chips with Bayesian hints, occasion/keyword chips, "what the AI will use" panel
Generation pipeline	100%	12 pipeline + API	6 candidates over-generated → hard-constraint filter → dedup → MMR → top 3; 202 + polling pattern
LLM adapters	100%	Unit + integration	Template (default, offline), OpenAI, Anthropic, Ollama; retry 1/3/9 s + jitter, circuit breaker, tolerant JSON parse, cost metering
Review & compare	100%	Browser-verified	3 ranked cards, score rings, confidence chips, similarity meters, why-ranked text, expandable feature bars, radar compare, PCA map, filtered-candidate disclosure
Candidate actions	100%	API + browser	Select, edit, copy, like, dislike, reject-with-reason, regenerate — each writes a feedback event
Personalization	100%	Math golden tests	Preference vector (EMA), Beta arms, learned length stats, online weight updates, reason nudges, learned-values UI with reset
Calendar	100%	Scheduler tests	Month grid with status chips, cycle detail modal, generate-now/skip/reschedule, signed .ics subscription
History & search	100%	Service tests	Text search + semantic (embedding) search, filters, detail drawer with batch context, reuse
Analytics	100%	Service tests	Usage, Content, Quality tabs; Wilson CI bars, chi-square panel, novelty trend, entropy, MRR, correlation
Notifications	100%	Scheduler tests	In-app notifications + email (console backend in dev), reminder schedule
Settings	100%	UI verified	Voice, banned words, emoji, length, lead time, variety slider, pause learning, audience CRUD, export, delete

4. Quality assurance

Suite	Count	Coverage
<code>test_math.py</code>	35	Every golden value in Section 17: feature/score vector, gradient steps (w_N→0.161, w_P→0.195, w_L→0.094), MMR order [A,C,F], Wilson CI (3,10)=(0.108,0.603), choice probabilities ≈[0.46,0.34,0.20]
<code>test_pipeline.py</code>	12	Hard constraints, length limits, banned words, dedup, arm-aware MMR diversity, seeded confidence, stage payload shape
<code>test_api.py</code>	journey	Register → plan → generate (202 + poll) → feedback → approve → analytics, run against a live TestClient
<code>test_scheduler.py</code>	4	Reminder days, idempotent cycle preparation, catch-up after missed runs, plan auto-extension
Total	62	All passing

Gate	Result
Backend tests	62/62 pass
TypeScript typecheck	0 errors
ESLint (via next build)	0 warnings
Production build	15 routes built, ~88 kB shared JS
Notebooks	4/4 execute without error
Browser end-to-end	Login → dashboard → review → like → 200 OK confirmed live
CI pipeline	GitHub Actions: pytest on 3.10 + 3.11, frontend typecheck + build

5. Bugs found and fixed during development

A deliberately test-driven build: these were caught by the golden tests or live verification and fixed.

Issue	Resolution
SQLAlchemy enum column receiving a Python enum object	Pass the portable string value through the <code>SAEnum</code> converter
FastAPI asserted on <code>-> None</code> response annotations	Removed the annotations (204 responses)
Naive/aware datetime mixing from SQLite	Added a <code>UTCDateTime</code> TypeDecorator, applied to 20 columns
ICS generation overflowed the hour on all-day events	Replaced <code>start + timedelta(hours=1)</code> with a bounded end time
Generated candidates were never embedded → similarity always 0	Embed each candidate in <code>_make_candidate</code>
Ambiguous truth value on a NumPy blob comparison	Explicit <code>is not None</code> guard (fixed twice — regression caught by tests)
<code>data_dir</code> resolved to <code>backend/data</code> instead of repo root	Anchored to the repository root in both config and templates (fixed twice)
Cycle horizon off-by-one when filling 12 cycles	Loop to <code>CYCLE_HORIZON+1</code> , break when full
MMR could return two candidates from the same style arm	Added a <code>groups</code> parameter preferring unrepresented arms
Weight sanitizer did not converge to bounds	Iterative clip + renormalise loop
Frontend: 4 TypeScript errors, 18 unused-import lint errors	Fixed; typecheck and build now clean
Live-session bugs	
Review screen showed "nothing to review" despite an awaiting cycle	A stale pre-restart uvicorn process was serving old code; restarted with current code
Silent token refresh failed after 15 minutes → 401 on feedback	API base used <code>127.0.0.1</code> while the page is <code>localhost</code> (different <i>site</i> , so the <code>SameSite=Lax</code> refresh cookie was never sent). Switched to <code>localhost</code> and made the cookie <code>None+Secure</code> under <code>MM_ENVIRONMENT=production</code> for cross-domain hosting

6. Documented deviations from the blueprint

Blueprint recommendation	Shipped instead	Reason
PostgreSQL + pgvector	SQLite default, Postgres-ready	No Docker/Postgres on the target machine; one env var switches and the schema + tests are identical
sentence-transformers embeddings	Own 384-d hashing embedder, MiniLM optional	Instant startup, offline, deterministic; same dimensionality so every formula and golden test is unchanged
spaCy + YAKE preprocessing	Own <code>text.py</code> (spaCy as an optional extra)	Heavy native dependencies; the needed operations are small, testable in isolation
shadcn/ui components	Own component library on Tailwind tokens	Fewer moving parts, exact control of the design tokens, no CLI-generated drift
Hosted LLM in the hot path	Template engine as the default provider	Zero-cost, always-available demo; hosted adapters retain retry, breaker and cost metering for production

7. Repository & deliverables

- **GitHub:** <https://github.com/vxrsha1501/monthly-muse> — branch `main`, 119 files, clean working tree.
- **Documentation:** README (quickstart + config matrix), `docs/api.md` (generated from the live route table), `docs/tech-stack.pdf` (9-page stack reference), this report.
- **DevOps:** `backend/Dockerfile`, `frontend/Dockerfile`, `docker-compose.yml` (Postgres + API + worker + frontend), `.github/workflows/ci.yml`.
- **Seed data:** 60 occasions, 20 topics, 40 templates, 800-row tone training set, and a demo account (`demo@monthlymuse.app` / `monthlymuse-demo`) with two published months of history.
- **Notebooks:** cosine calibration, tone classifier, ablation study, simulated users — all execute.

8. Deployment readiness

The application is architecture-ready for a live deployment with **no code changes** — only environment variables.

Piece	Target	Notes
Database	Render Postgres (or Neon)	Set <code>MM_DATABASE_URL</code> to the managed connection string; schema is already portable
API service	Render web service (Docker)	Start command <code>uvicorn app.main:app --host 0.0.0.0 --port \$PORT</code> ; set <code>MM_ENVIRONMENT=production</code> and a strong <code>MM_SECRET_KEY</code>
Worker service	Render web service (Docker)	<code>while true; do python -m app.scheduler.runner; sleep 300; done</code> so free-tier sleep cannot skip the daily run
Frontend	Vercel or Render	Set <code>NEXT_PUBLIC_API_URL</code> to the live API origin
Secrets	Platform env vars	Never committed; the repo contains no secrets

Known constraint: free-tier web services sleep after ~15 minutes of inactivity, which is why the worker runs in a restart loop — the scheduler's daily preparation is idempotent, so a sleeping and waking worker cannot duplicate a cycle or a reminder.

9. Remaining work

Item	Priority
Perform the live deployment (Render + Postgres + Vercel) and verify the demo account online	Next — accounts exist, configuration is ready
Add a Playwright end-to-end test covering login → review → select → analytics	Medium — the API journey is already covered
Connect a hosted LLM key (OpenAI / Ollama) so candidates are LLM-written	Medium — adapters are built and tested; only the key and env var are needed
Polish offline template wording (minor grammatical artefacts in generated copy)	Low — affects the template fallback only
Version 2 items (\$15.2): platform publisher, engagement import, teams, dark mode, autopilot	Out of MVP scope